team@hackathon:~$

COURSE · 완전 초보를 위한 바이브코딩

기획 문서를, 클릭 가능한 링크로.

무료 AI + Cursor로 HTML 데모를 만들고
GitHub Public · Vercel 배포까지

Max · AI 바이브코딩 실습 강의

◎ MISSION

기획을 클릭 가능한 링크로

01 · THINK

기획 문서

문제 · 사용자 · 핵심 흐름을
DEMO_PLAN.md 한 장으로

→

02 · BUILD

HTML 데모

AI가 index.html 제작
내가 버튼을 직접 클릭

→

03 · SHARE

공개 URL

GitHub에 보관하고
Vercel로 누구나 접속

완성 기준: 휴대전화에서 URL을 열고, 우리 아이디어의 핵심 흐름을 직접 눌러볼 수 있다.

PLAN.md 계획 문서 · 만들 기능과 범위를 미리 정리해 두는 파일 **URL** Uniform Resource Locator · 웹 주소

ROADMAP · 8월 14일 → 25일

10일 준비하고, 2일 집중 구현

8.14–8.18

아이디어를 눈에 보이게

기획 문서 정리
Manus·Cursor로 HTML 데모
팀 안에서 클릭 검증

결과 · index.html

8.19–8.23 · 본편

링크로 공유하고 반응을 관찰

GitHub Public 업로드
Vercel URL 만들기
팀 밖 3명에게 테스트

결과 · 공개 URL

8.24–8.25 · 부록

본캠프에서 실제 기능 구현

Claude Code Max
핵심기능 · 데이터 · 검증
발표·영상·최종 제출

결과 · 작동 서비스

오늘은 **URL 하나를 만드는 데 집중**한다. 본캠프 작업은 뒤쪽 **부록**에서 찾아본다.

URL Uniform Resource Locator · 웹 주소

FREE FIRST · 결제 없이 시작

무료로도 준비는 충분히 가능하다

생각 · 무료

Claude
ChatGPT

질문 → 문제 정의
기획·문구·오류 설명

→

연습 · 무료 한도

Manus
Cursor

기획 → HTML 데모
파일 생성·수정 경험

→

본캠프 · 팀 계정

Claude Code
Max

실제 기능 · 테스트
Git·배포 반복 자동화

추천 경로: 무료 채팅 AI로 명세 작성 → **Cursor Free로 작은 기능 1개** → 본캠프 Max로 핵심기능 3개 완성.

Cursor Hobby Free · Claude Code 플랜 · ChatGPT 공식 안내 · 2026-08-12 기준

STEP 1 · PLAN

먼저 DEMO_PLAN.md 한 장

DEMO_PLAN.md

1. 서비스 한 문장
2. 대표 사용자
3. 해결할 문제
4. 핵심 흐름 3단계
5. 화면 문구·예시 데이터
6. 디자인 분위기

무료 CLAUDE · CHATGPT

"우리 아이디어를 **클릭 가능한 HTML 데모**로 만들 준비를 해줘. 질문이 필요하면 한 번에 하나씩 물어보고, 답변을 바탕으로 DEMO_PLAN.md를 작성해줘.

서비스 한 문장, 대표 사용자, 해결할 문제, 사용자가 누르는 핵심 흐름 3단계, 화면 문구와 예시 데이터, 원하는 디자인 분위기를 포함해줘."

완료 팀원 모두가 읽고 **"이 화면이면 아이디어가 설명된다"**고 동의

PLAN.md 계획 문서 · 만들 기능과 범위를 미리 정리해 두는 파일

MANUS · 설치 없이 브라우저에서

사전 연습 1순위

행사 안내 기준 · 8월 25일까지

기획을 넣고 데모부터 만들어 본다

Manus는 브라우저에서 요청을 받아 자료 조사·문서 작성·웹페이지 제작 같은 작업을 대신 수행하는 **AI 에이전트**입니다. 설치 없이 기획서를 HTML 데모로 바꾸는 연습에 적합합니다.

01 · SIGN IN

무료로 시작

manus.im 접속

계정 생성 · 잔여 크레딧 확인

→

02 · BUILD

기획서 전달

DEMO_PLAN.md 붙여넣기

HTML 데모 제작 요청

→

03 · EXPORT

파일 가져오기

결과 확인 · 수정 반복

index.html 다운로드

Manus 실행하기 ↗

다음 파일 확인 → **GitHub Public → Vercel**

Manus 시작하기 · 공식 Free 플랜 안내 · 무료 기간·크레딧은 계정 화면에서 재확인

PLAN.md 계획 문서 · 만들 기능과 범위를 미리 정리해 두는 파일

CURSOR · HOBBY FREE

사전 10일 추천

무료로 실제 파일을 수정해 본다

```
Cursor
├── 프로젝트 폴더 읽기
├── Agent에게 한국어로 요청
├── 파일 생성·수정 제안
├── 코드 실행·오류 질문
└── 변경 내용 직접 확인
```

- ✓ **신용카드 없이** Hobby Free 시작
- ✓ 제한된 Agent 요청 제공
- ✓ Composer 사용 가능
- ✓ 무료 한도는 작게 나눠 사용

목표는 완성품이 아니다. 사전에는 랜딩 페이지 또는 핵심기능 1개를 직접 만들고, AI에게 요청하고 검증하는 흐름을 익힙니다.

Cursor 공식 다운로드 · Hobby Free 플랜 확인

CURSOR · 설치 5분

다운로드 → 로그인 → 폴더 열기

01 · DOWNLOAD

공식 사이트

cursor.com/downloads

Windows 또는 Mac 선택

→

02 · SIGN IN

무료 가입

Hobby Free 시작

카드 등록 불필요

→

03 · OPEN

폴더 열기

Open project

team-project 선택

Mac

다운로드 → Applications로 이동 → 실행

Windows

설치 파일 실행 → 기본 옵션 → 실행

완료 왼쪽 파일 목록에 **team-project 폴더**가 보이면 성공

STEP 2 · CURSOR AGENT

기획 문서를 index.html로

CURSOR AGENT · 그대로 복사

"DEMO_PLAN.md를 읽고 **index.html 하나로 클릭 가능한 서비스 데모**를 만들어줘. CSS와 JavaScript도 파일 안에 포함해줘. 서버·데이터베이스·로그인·API·패키지 설치는 사용하지 마. 예시 데이터만 사용하고, 핵심 흐름 3단계의 버튼이 실제로 눌리게 해줘. 모바일에서도 보기 좋게 만들고 완료 후 브라우저에서 여는 방법을 알려줘."

1
파일 열기
index.html 더블클릭

→

2
흐름 누르기
버튼 3단계 확인

→

3
모바일 확인
창 폭을 줄여보기

무료 한도 절약 공식 — 기획은 채팅 AI · 구현은 Cursor · 한 번에 기능 하나

PLAN.md 계획 문서 · 만들 기능과 범위를 미리 정리해 두는 파일

API Application Programming Interface · 프로그램끼리 데이터를 주고받는 통로

GITHUB · 코드 안전망

8월 23일까지 필수

본캠프 전에 가입 + 첫 백업

01 · ACCOUNT

계정 만들기

github.com/signup

이메일 인증까지

→

02 · REPOSITORY

저장소 생성

team-project

Public 선택

→

03 · PUSH

첫 백업

커밋 후 push

웹에서 파일 확인

CURSOR에게 요청

"이 프로젝트를 Git 저장소로 만들고 첫 커밋을 해줘. **Public 저장소이므로 .env·API 키·비밀번호·개인정보가 올라가지 않게 전체를 점검**하고 .gitignore를 확인해줘. 저장소 연결은 한 단계씩 설명해줘."

완료 GitHub에서 코드가 보이고 · **.env는 안 보이고** · 저장소 URL을 팀방에 공유

GitHub 무료 가입 · GitHub Desktop 다운로드 · 팀당 최소 1계정

.env environment variables · 비밀값(키·비밀번호)을 코드와 분리해 담는 파일 **.gitignore** git ignore · 저장소에 올리지 않을 파일 목록(.env 등 보호)

VERCEL · 첫 공개 URL

8월 23일까지 필수

GitHub 연결하고 Deploy

01 · SIGN UP

GitHub로 가입

vercel.com/signup

Continue with GitHub

→

02 · IMPORT

저장소 연결

Add New → Project

team-project Import

→

03 · DEPLOY

배포

설정 확인 → Deploy

공개 URL 복사

비밀값이 있다면: GitHub에 올리지 말고 Vercel **Project → Settings → Environment Variables**에 입력 → 새로 Deploy.

완료 휴대전화 또는 시크릿 창에서 ***.vercel.app URL**이 열리고 핵심 화면이 보임

Vercel 무료 가입 · Git 저장소 배포 공식 가이드 · push할 때마다 자동 재배포

URL Uniform Resource Locator · 웹 주소

MENTAL MODEL · 저장과 공개의 차이

GitHub는 보관함, Vercel은 전시장

내 컴퓨터

index.html

AI와 만들고
직접 클릭해 확인

push →

GITHUB PUBLIC

코드 보관

변경 이력과 파일을
인터넷에 백업

연결 →

VERCEL

웹으로 배포

코드를 실행 가능한
사이트로 변환

URL →

사용자

휴대전화

설치 없이 링크를 열고
데모를 직접 클릭

기억 **GitHub URL = 코드 보기 · Vercel URL = 서비스 사용**

URL Uniform Resource Locator · 웹 주소

WORKFLOW · 무료 채팅 AI 활용법

도구보다 중요한 것은 남는 문서

01 · CONTEXT

맥락 제공

누구의 어떤 문제인지
현재 결정된 내용 첨부

→

02 · ASK

작게 요청

목표 + 제약 +
완료기준을 한 번에

→

03 · SAVE

문서로 저장

결정 내용을 PLAN.md와
화면 명세에 계속 누적

무료 Claude·ChatGPT 공통 프롬프트

"우리는 **[사용자]**의 **[문제]**를 해결하는 서비스를 준비 중이야. 핵심기능 후보를 5개 제안한 뒤, **10일 안에 검증 가능한 3개만** 골라줘. 각 기능을 '사용자 입력 → 화면 결과 → 완료기준'으로 정리해줘."

PLAN.md 계획 문서 · 만들 기능과 범위를 미리 정리해 두는 파일

CHECKLIST · 오늘의 완주선

링크가 열리면 완료

- ✓ **DEMO_PLAN.md 완성**
- ✓ **index.html 생성**
- ✓ 핵심 흐름 3단계 클릭
- ✓ 모바일 화면 확인
- ✓ 가짜 예시 데이터만 사용
- ✓ GitHub Public 첫 push
- ✓ 비밀값·개인정보 없음
- ✓ **Vercel 공개 URL 생성**
- ✓ 휴대전화에서 URL 확인
- ✓ 두 URL을 팀방에 공유

완료 팀원이 아닌 사람도 URL을 열고 **아이디어의 핵심 흐름을 이해**할 수 있음

PLAN.md 계획 문서 · 만들 기능과 범위를 미리 정리해 두는 파일 **URL** Uniform Resource Locator · 웹 주소

OVERVIEW · 오늘 가져갈 것

오늘의 핵심 5

1 **기획 문서를 한 장으로** — DEMO_PLAN.md

2 **Cursor로 HTML 데모** — index.html 파일 하나

3 **내 눈으로 직접 클릭** — 핵심 흐름 3단계 검증

4 **GitHub Public에 push** — 비밀값·개인정보는 제외

5 **Vercel URL 공유** — 휴대전화에서 열리면 완료

권고 — 오른쪽 위 **'권고'** 칩은 전략·조언 표시 · 이 자료는 끝나고 전체 공유

1시간 뒤, 이 다섯 줄을 다시 만난다

PLAN.md 계획 문서 · 만들 기능과 범위를 미리 정리해 두는 파일 **URL** Uniform Resource Locator · 웹 주소

team@hackathon:~$



I · MINDSET

코딩 몰라도 됩니다,
만들 것을 알면 됩니다

MINDSET · VIBE CODING

바이브 코딩 — 의도를 말하면 AI가 코드를 쓴다.
무엇을·왜는 여러분, **어떻게**는 AI. 그리고 그 방식으로 —

"코드를 **한 줄도 안 썼고**,
한 줄도 **안 읽었다**."

— Mike Brown · Anthropic 공식 해커톤 1위 (주택 인허가 자동 검토 서비스)

4 / 5

수상자 중 **비개발자** — 변호사 · 의사 · 인프라 전문가 · 뮤지션

코드가 평준화되면, 차별화는 **도메인 지식과 고객 이해**에서 나온다 — 창업팀인 **여러분의 경쟁 우위**

Andrej Karpathy 2025-02 명명 · Collins '올해의 단어' · claude.com/blog 해커톤 수상자 발표

MINDSET · 단 하나의 규칙

AI를 믿지 말고, 확인하라



규칙: 기능 하나 = **"만들어졌다" + "내가 직접 눌러봤다"** 세트로 한 단위. 확인 없이 다음 기능으로 넘어가지 않는다 — 오늘 배울 모든 것의 기반.

Stack Overflow Developer Survey 2025 · n≈49,000 · "almost right, but not quite"

DONE · 본편 완료

기획이 이제 URL이 되었다

PLAN

DEMO_PLAN.md

아이디어와 사용자 흐름

→

DEMO

index.html

클릭 가능한 화면

→

SHARE

Vercel URL

누구나 여는 링크

8월 23일까지 할 일

팀 밖의 사람 3명에게 URL만 보내고 사용하게 해보세요. 설명 없이 이해하지 못한 지점을 기록해 DEMO_PLAN.md와 index.html을 고칩니다.

실제 기능 구현 · Claude Code Max · 본캠프 운영은 다음 장부터 부록

PLAN.md 계획 문서 · 만들 기능과 범위를 미리 정리해 두는 파일 **URL** Uniform Resource Locator · 웹 주소

team@hackathon:~$

APPENDIX · BOOTCAMP

여기부터 부록 — 본캠프에서 펼쳐보는 실전 가이드

Claude Code 설치 · Max 로그인 · Git · 핵심기능 구현 · 최종 제출

BEGINNER · 먼저 구분하기

Claude와 Claude Code는 뭐가 다를까?



대화하고 답을 받는

Claude

"홈페이지를 어떻게 만들면 좋을까?"
→ 설명·문구·코드를 **답변으로 제공**



내가 직접 복사하고 파일에 붙여넣음

VS

같은 두뇌
다른 손발



폴더에서 행동하는

Claude Code

"홈페이지를 만들어줘."
→ 폴더를 읽고 **파일을 직접 생성·수정**



index.html이 생기고 실행·검증까지 도움

한 줄 차이: Claude는 "방법을 알려준다" · Claude Code는 내 폴더에서 "실제로 바꾼다".

공통점: 같은 Claude 모델 계열 · 차이점: 연결된 도구와 작업 범위

BEGINNER · 실제 예시로 이해하기

같은 요청인데, 결과가 다르다

Claude에게 말하면

"로그인 버튼을 빨간색으로 바꿔줘."

① 바꿀 CSS 코드를 알려줌
② 어느 파일인지 **내가 찾아야 함**
③ 코드를 **내가 복사·붙여넣기**
④ 작동 확인도 내가 진행

Claude Code에게 말하면

"로그인 버튼을 빨간색으로 바꿔줘."

① 프로젝트에서 버튼을 직접 찾음
② 바꿀 내용을 보여주고 승인 요청
③ **파일을 직접 수정**
④ 실행·테스트 방법까지 안내

선택법 **생각·질문·기획**은 Claude · **파일 생성·수정·실행**은 Claude Code

※ Claude **데스크톱 앱의 Code 화면**을 열면, 앱 안에서도 Claude Code 기능을 사용할 수 있습니다.

SETUP · 설치 전 체크

시작 전에 3가지만 확인

① 컴퓨터

Windows 10+ 또는 **macOS 10.15+**
메모리 4GB 이상

② 인터넷

설치 파일 다운로드와
로그인·AI 처리를 위해 필요

③ 계정

Claude Pro/Max 또는
조직 계정·Console API 계정

초보자 권장: API 키를 직접 넣지 말고, 설치 후 **claude**를 실행해 브라우저에서 Claude 계정으로 로그인하세요.

Anthropic 공식 설치 문서 · 자료 확인일 2026-08-12

API Application Programming Interface · 프로그램끼리 데이터를 주고받는 통로

SETUP · MAC

MAC 전용

Mac 설치 — 그대로 따라하기

1 터미널 열기

⌘ Command + Space → “터미널” 검색 → Enter

2 공식 설치 명령 붙여넣기

```
$ curl -fsSL https://claude.ai/install.sh | bash
```

3 터미널 완전 종료 후 확인

```
$ claude --version
$ claude
```

절대 붙이지 않기: sudo — 나중에 권한 오류가 생길 수 있습니다.

SETUP · WINDOWS

WINDOWS 전용

Windows 설치 — Git이 먼저

1 · Git for Windows

```
> winget install --id Git.Git -e
```

또는 **git-scm.com**에서 다운로드 → 기본값으로 설치

2 · Claude Code

```
> irm https://claude.ai/install.ps1 | iex
```

대안: `winget install Anthropic.ClaudeCode`

```
# PowerShell을 완전히 닫고 다시 연 뒤
> git --version
> claude --version
> claude
```

관리자 PowerShell 불필요. 회사 PC에서 스크립트가 차단되면 **winget 설치 경로**를 사용하세요.

git version control · 코드 변경 이력을 저장·복원하는 도구

GIT · 최초 1회 설정

Git에게 “누가 저장했는지” 알려주기

```
# 따옴표 안을 내 정보로 변경
$ git config --global user.name "Max Seo"
$ git config --global user.email
"me@example.com"
$ git config --global init.defaultBranch main

# 제대로 저장됐는지 확인
$ git config --global --list
```

이메일 공개가 걱정되면 GitHub Settings → Emails의 **noreply 이메일** 사용

쉽게 기억하기

Git = 내 컴퓨터의 저장 이력

커밋 = 게임의 세이브 포인트

GitHub = 인터넷 백업 창고

TROUBLESHOOT · WINDOWS + GIT

WINDOWS 오류

“Git Bash를 못 찾습니다”

git을 찾을 수 없음	Git for Windows 설치 → **PowerShell 완전 종료·재실행** → `git --version`
Claude가 bash.exe를 못 찾음	`$env:CLAUDE_CODE_GIT_BASH_PATH="C:\Program Files\Git\bin\bash.exe"`
설정이 재실행 후 사라짐	`setx CLAUDE_CODE_GIT_BASH_PATH "C:\Program Files\Git\bin\bash.exe"` → 새 PowerShell
경로에 파일이 없음	파일 탐색기에서 **Git\bin\bash.exe** 실제 위치 확인 후 그 경로 사용

git version control · 코드 변경 이력을 저장·복원하는 도구

TROUBLESHOOT · MAC

MAC 오류

Mac에서 가장 자주 막히는 곳

command not found: claude	`echo 'export PATH="$PATH:$HOME/.local/bin"' >> ~/.zshrc` → 터미널 재시작
git: command not found	`xcode-select --install` → 설치 창 완료 → 터미널 재시작
npm EACCES 권한 오류	**sudo로 반복하지 않기.** 공식 네이티브 설치 명령 사용 또는 `claude migrate-installer`
다운로드·인증서·접속 오류	VPN 해제 또는 다른 네트워크 시도 · 회사 PC라면 보안 정책 담당자 확인

TROUBLESHOOT · LOGIN & BILLING

로그인은 됐는데, 계정이 이상하다?

1

브라우저가 안 열림

터미널에 나온 로그인 URL을 복사해 Chrome 주소창에 직접 붙여넣기

2

원하는 Pro/Max 계정이 아님

```
> /logout   → Claude Code 종료 → claude update → 터미널 재시작 → claude
```

3

예상치 못한 API 요금

ANTHROPIC_API_KEY가 설정되어 있으면 구독 대신 API 과금이 사용될 수 있음

초보자 원칙: 행사·개인 Pro/Max를 쓸 때는 API 키를 새로 만들지 말고 **브라우저 로그인**을 선택합니다.

URL Uniform Resource Locator · 웹 주소 **API** Application Programming Interface · 프로그램끼리 데이터를 주고받는 통로

CHECKPOINT · 설치 완료

이 화면까지 되면 진짜 완료

- ✓ `claude --version` → 버전 번호가 나온다
- ✓ `git --version` → Git 버전 번호가 나온다
- ✓ `claude doctor` → 치명적인 설치 오류가 없다
- ✓ `claude` 실행 → 브라우저 로그인 → 한국어 답변이 온다
- ✓ `git config --global --list` → 내 이름과 이메일이 보인다

완료 기준 **5개 모두 체크**하고 프로젝트 폴더에서 Claude Code 시작

문제 발생 시 오류 메시지를 요약하지 말고 전체를 복사해 Claude 또는 멘토에게 전달

git version control · 코드 변경 이력을 저장·복원하는 도구

SETUP · 준비물 & 계정

본캠프 — 팀당 Max 1개

- ✓ **노트북 + 충전기** — 배터리로 해커톤을 버틸 수 없다
- ✓ **8월 24일 — 주최측 제공 Max 계정 수령** (팀당 1개, 개인 결제 불필요)
- ✓ **Chrome 브라우저** — 로그인·미리보기·배포 전부 여기서
- ✓ **GitHub 계정 — 팀당 1개 이상** (무료 가입 → 이메일 인증까지)
- ✓ **Vercel 계정 — GitHub로 가입·연결** (배포용)

Max 로그인은 메인(드라이버) 노트북 1대에만 — 한도는 팀의 공유 자원. 닿으면 5시간 후 리셋(그때가 수면·회의 타이밍) · 개인 Pro 병행은 자유.

GitHub 가입 · Vercel 가입 · 계정 회수 전 코드는 GitHub에 백업

SETUP · 같은 출발선 만들기

초보자 · 1/3

로그인 → 환경 선택 → 폴더 열기

01 **본캠프 당일 · Max 로그인** 5분

팀 메인 노트북에서 로그인 → `/model` 확인 → `/usage` 위치 확인

02 **작업 환경 하나만 선택** 2분

완전 초보 = Claude 앱 · 터미널 경험자 = 터미널 · 기존 IDE 사용자는 그대로

03 **영문 프로젝트 폴더 생성** 3분

`Desktop/team-project/` 생성 → Claude에서 해당 폴더 열기 · 모든 자료와 코드는 여기만

완료 Claude가 **team-project 폴더를 읽고 답변**하면 다음 단계

설치 개수보다 중요한 것 — 전원이 같은 폴더에서 첫 답변 받기

IDE Integrated Development Environment · 코드를 작성·실행하는 통합 개발 프로그램

SETUP · 준비된 기획을 개발 규칙으로

기획서 전달 → CLAUDE.md

폴더에 넣기 · 3분

```
team-project/
├── market-research.pdf
├── service-plan.pdf
└── core-features.md
```

다시 기획하지 않는다.
준비한 기능을 구현 언어로만 변환.

한 번 복사 · 5분

PROMPT · 자료 첨부 후 전송

첨부한 기획서와 핵심기능 3가지를 읽어줘.

CLAUDE.md를 만들고 저장해줘:
한국어로 답하기 · 기능은 한 번에 하나 · 수정 전 순서 설명 · 기존 기능 임의 변경 금지 · 완료 후 직접 실행·검증 · 기능마다 git 커밋 · 새 기능 추가 금지.

완료 **CLAUDE.md 파일 생성** + Claude가 준비된 핵심기능 3가지를 정확히 요약

추가 스킬·플러그인 설치 없음 — 작업 규칙은 CLAUDE.md 한 장

CLAUDE.md 프로젝트 규칙 문서 · Claude에게 매번 줄 지시를 자동 적용 **git** version control · 코드 변경 이력을 저장·복원하는 도구

SETUP · 코드 잃어버리지 않기

초보자 · 3/3

GitHub 가입 → 첫 백업

01 · SIGN UP
GitHub 가입
github.com/signup
이메일 인증까지

→

02 · PUBLIC
저장소 생성
`team-project`
Public 선택

→

03 · BACKUP
Claude에게 요청
Git 초기화 · 보안 확인
커밋 · GitHub push

PROMPT · 저장소 주소를 붙여 전송

"이 프로젝트에 git 저장소를 만들어줘. **Public 저장소이므로 .env·.env.local·API 키·개인정보가 올라가지 않도록 점검**하고 첫 커밋을 해줘. 그다음 저장소에 연결해 push해줘: **[GitHub 저장소 주소]**"

커밋	GitHub	Public
코드 저장 지점	노트북 밖 외부 백업	누구나 볼 수 있음

완료 GitHub에 코드는 보이고 **.env는 안 보임** → 저장소 URL을 팀방에 공유

GitHub 가입 · 팀당 최소 1계정 · 로그인 승인 창은 팀 계정으로 진행

.env environment variables · 비밀값(키·비밀번호)을 코드와 분리해 담는 파일 **URL** Uniform Resource Locator · 웹 주소

SETUP · 기간별 도구 선택

사전에는 Cursor, 본캠프는 Claude 앱

권고

8.14–23 · 기본 경로

Cursor Hobby Free

무료로 파일 수정 경험 — 기획서를 읽히고 작은 기능 1개를 직접 구현

무료 Claude · ChatGPT

기획·명세·오류 질문 — 결정한 내용은 PLAN.md에 저장해 도구 간 이동

8.24–25 · 초보자 추천

Claude 데스크톱 앱

익숙한 채팅 화면에서 Code 실행 — 폴더 선택 후 파일 수정·명령 실행

초보자는 Claude 앱 추천 · Cursor 안의 Claude Code도 가능 · 터미널은 선택.

Cursor 다운로드 · Claude 앱 다운로드 · Claude Code 명령어는 터미널 기준

PLAN.md 계획 문서 · 만들 기능과 범위를 미리 정리해 두는 파일

CLAUDE DESKTOP · 터미널 불필요

완전 초보 추천

Claude 앱 설치 — Windows · Mac

Claude 공식 앱 다운로드 ↗

Windows

1. **Download for Windows** 선택
2. 내려받은 설치 파일 실행
3. 설치 후 시작 메뉴에서 **Claude** 실행
4. Max 로그인 → **Code** → team-project 선택

Mac

1. **Download for macOS** 선택
2. 파일을 열고 Claude를 **Applications**로 이동
3. 응용 프로그램에서 **Claude** 실행
4. Max 로그인 → **Code** → team-project 선택

완료 앱의 Code 화면에서 **Cursor로 준비한 같은 폴더**가 열리면 성공

Claude 공식 앱 다운로드 · Code 세션을 지원하는 최신 앱 사용

CURSOR + CLAUDE CODE

Cursor 사용자 추천

Cursor를 그대로 쓰고, Claude Code 추가

1 **Cursor에서 Extensions 열기**

왼쪽 확장 아이콘 또는 Mac ⇧⌘X · Windows Ctrl+Shift+X

2 **Claude Code 검색·설치**

게시자가 **Anthropic**인지 확인하고 Install

3 **Max 로그인 후 현재 폴더에서 시작**

Claude Code 패널 열기 → 제공 계정 로그인 → 준비한 기능 ① 요청

혼동 주의: Cursor Agent에서 “Claude 모델”을 고르는 것과 **Anthropic Claude Code 확장**을 설치하는 것은 다릅니다.

Claude Code IDE 지원 안내 · Cursor 및 VS Code 계열 지원

IDE Integrated Development Environment · 코드를 작성·실행하는 통합 개발 프로그램

SETUP · 터미널 설치

선택 · 터미널 사용자

터미널 방식은 선택 사항

● MAC — ⌘+Space → "터미널" → Enter

```
$ curl -fsSL
https://claude.ai/install.sh | bash
```

● WINDOWS — 시작 → "PowerShell" → Enter

```
> irm https://claude.ai/install.ps1 |
iex
```

공통 · 건너뛰면 안 되는 한 가지: 설치가 끝나면 터미널을 **완전히 종료(⌘Q)했다가 다시 열기** — 재시작해야 **claude** 명령을 찾는다.

Node.js 등 사전 설치 불필요 · Windows에서 git 안내가 나오면 **git-scm.com** 설치(전부 Next) 후 PowerShell 재시작

git version control · 코드 변경 이력을 저장·복원하는 도구

SETUP · STEP 2

실행하고, 로그인

1 터미널에 입력

```
$ claude
```

2 8월 24일 — 제공된 Max 계정으로 로그인

메인 노트북 1대에서 승인 → 터미널 복귀 · 사전 기간에는 유료 접근이 있는 학생만 선택 실습

3 성공 확인 — 한국어로 인사

입력창에 **"안녕? 넌 뭘 할 수 있어?"** → 답이 오면 **세팅 성공**

심화 · 선택

세팅에 익숙해지면

orca

여러 에이전트 세션과 작업 폴더를 한 화면에서 관리 · 데스크톱 앱

cmux

여러 에이전트를 동시에 병렬 실행 · macOS 네이티브 터미널 (Mac 전용)

모든 팀의 메인 노트북에서 확인될 때까지 함께 진행합니다 — 오늘의 첫 체크포인트

SETUP · 트러블슈팅

안 될 때 — 즉효 처방

"command not found: claude"	**터미널 완전 종료 → 다시 열기** — 원인의 90%. Mac은 ⌘Q까지
붙여넣기가 안 됨	Mac **⌘V** · Windows **우클릭** 또는 Ctrl+V · 안 되면 직접 타이핑
로그인 브라우저가 안 열림	터미널에 표시된 **URL을 복사**해서 브라우저 주소창에 직접 붙여넣기
(Windows) "git이 필요합니다"	**git-scm.com** → Git for Windows 설치(전부 Next) → PowerShell 재시작
그래도 안 됨 · 2분 경과	**스태프·멘토 호출** — 붙잡고 있는 시간이 제일 비싸다

URL Uniform Resource Locator · 웹 주소 **git** version control · 코드 변경 이력을 저장·복원하는 도구

SETUP · STEP 3 · 5분 도전

첫 페이지 — 말로 만든다

```
# 팀 폴더 만들고 시작 (팀명은 영문 소문자 — 이 폴더가 해커톤 기간의 작업장)
$ mkdir teamrocket && cd teamrocket && claude
```

PROMPT · 빈칸만 채워서 그대로 전송

우리 팀 서비스 **"___(서비스 한 줄 소개)"**의 랜딩 페이지를 **index.html 한 파일**로 만들어줘. 제목, 한 줄 소개, 핵심 기능 3개 카드, "사전 신청" 버튼 포함. 다 만들면 **브라우저로 열어줘**.

- ✓ 뭘 해도 되냐고 물으면 — 읽고 **Enter(허락)** · 브라우저에 내 서비스가 표시되면 — **완료**
- ✓ 먼저 끝났다면 — **"문구 더 매력적으로 바꿔줘"** 추가 주문

방금 한 것 = 명세 → 생성 → 확인. 이 루프가 해커톤 내내 반복된다

SETUP · 조작법

해커톤 동안 쓸 조작 — 6개면 끝

Esc

이상한 방향으로 달리면 **즉시 중단** — 부담 없이 사용

/clear

기능 하나 끝날 때마다 **대화 초기화** — 긴 대화는 맥락 정확도가 떨어진다

스크린샷 붙여넣기

말 대신 이미지 — 캡처 후 **창에 드래그** · 붙여넣기는 **Ctrl+V (Mac도!)**

Shift+Tab

자동 승인 모드 — 일일이 허락 없이 진행. 대신 결과 검증은 더 철저히

Plan 모드

Shift+Tab **두 번** (또는 /plan) — 큰 작업 전 "계획 먼저"

"되돌려줘"

"마지막 잘 되던 상태로 되돌려줘" — git 커밋 기반 복원 기능

체크: claude 대화 ✓ 첫 페이지 ✓ 팀 폴더 ✓ — 핵심 2번 완료

git version control · 코드 변경 이력을 저장·복원하는 도구

SETUP · 명령어 사전

알아두면 좋은 명령어 6

`/init`	프로젝트를 분석해 **CLAUDE.md 자동 생성** — 새 프로젝트에서 1회
`/clear`	대화 **전체 초기화** — 새 기능을 시작할 때
`/compact`	대화를 **요약해 압축** — 맥락을 유지한 채 같은 작업을 이어갈 때
`/context`	**컨텍스트(대화 메모리) 사용 현황** — 가득 차기 전 /compact·/clear 판단 근거
`/usage`	**사용량·한도 확인** — Max 계정 잔여 한도 관리 (팀 공유 자원)

`claude --dangerously-skip-permissions` — 모든 승인 확인을 생략하는 **시작 옵션**. 빠르지만 위험하다 — 기본은 Shift+Tab으로 충분, **동작을 이해한 뒤 팀 폴더 안에서만**. (앱은 Mode selector → Bypass)

전체 목록은 `/help` · 앱의 슬래시 명령어 지원은 버전마다 다름

CLAUDE.md 프로젝트 규칙 문서 · Claude에게 매번 줄 지시를 자동 적용

SETUP · 컨텍스트 관리

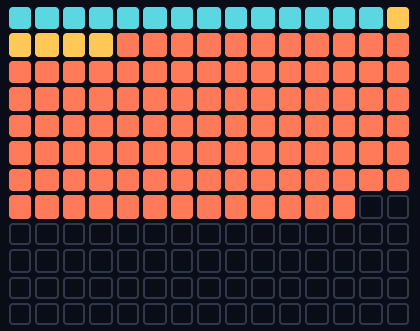
권고

컨텍스트 — AI의 작업 기억

토큰 = AI가 글을 읽는 **작은 조각**
글자 수·단어 수와 정확히 같지 않다

"로그인 기능을 만들어줘" → **로그인** · **기능** · **을** · **만들어** · **줘**
개념 예시 — 실제 분할은 모델·문장에 따라 다름

대화·코드·문서·AI 답변의 토큰이 쌓이는 공간이 **컨텍스트** — AI의 작업 기억.



/context

640k / 1M 토큰 · **64%**

- 시스템 · 도구 8%
- 스킬 · 메모리 3%
- 대화(Messages) 55% ← 대부분을 차지
- 남은 공간 34%

⚠ 꽉 차면 느려짐 · 잊음 · 엉뚱한 수정 · 반복

/context
얼마나 찼는지 **확인**

/compact
대화를 **요약·압축**

/clear
초기화 — 새 기능 시작

규칙 — 기능 하나 끝나면 /clear · 길게 이어가야 하면 /compact · 답이 느려지면 /context로 점검

SETUP · 토큰은 어디서 쌓이나

개념

입력도, 출력도 — 모두 컨텍스트

한국어는 몇 토큰?

한국어 1글자	≈ 1토큰 안팎
한국어 100글자	≈ 70–150토큰

모델·문장에 따라 달라지는 넓은 범위

정확한 공식은 없다.
영어·숫자·이모지·코드·띄어쓰기가 섞이면 달라진다.

컨텍스트에 들어가는 것

- 시스템 · CLAUDE.md
- 이전 질문 · 이전 답변
- 코드 · 문서 · 이미지
- 도구 설명 · 실행 결과
- 현재 사용자 입력

→

INPUT
현재 요청에 함께 전달

+

OUTPUT
새 답변을 만들 공간

핵심 생성된 **출력**도 다음 질문에서는 **입력 기록**이 된다 → 대화가 길수록 계속 쌓인다

추천 **Caveman 스킬** — 기술 내용은 유지하고 답변을 압축해 토큰 절약 · 처음은 `/caveman lite`

정확한 토큰 수는 모델의 실제 usage로 확인 — 숫자는 이해를 위한 대략적인 범위

CLAUDE.md 프로젝트 규칙 문서 · Claude에게 매번 줄 지시를 자동 적용

team@hackathon:~$

III · WORKFLOW & SECURITY

작업 규칙과 보안 — 설치 없이 바로 적용한다

✧ SKILLS · 개념

권고

스킬 = 매뉴얼, 에이전트 = 팀원



이번 해커톤은 설치형 스킬 대신 **CLAUDE.md에 작업 규칙을 저장** — 팀마다 같은 절차로 바로 시작한다.

스킬 = 작업 절차 문서 · 에이전트 = 병렬 작업 보조 AI

CLAUDE.md 프로젝트 규칙 문서 · Claude에게 매번 줄 지시를 자동 적용

WORKFLOW · 설치 0개

CLAUDE.md — 한 번 저장하면 자동 적용

PROMPT · 시작할 때 한 번

"CLAUDE.md를 만들고 저장해줘: 항상 한국어로 답하기 · 기능은 한 번에 하나 · 수정 전 작업 순서 설명 · 완료 후 브라우저에서 직접 검증 · 기존 기능 임의 변경 금지 · 기능마다 git 커밋 · 새 기능 추가 금지. 저장한 뒤 준비된 기능 ①부터 구현해줘."

① 설치가 없다

플러그인·마켓플레이스 없이 **텍스트 파일 한 장**으로 바로 시작

② 매번 자동 적용

프로젝트를 다시 열어도 Claude가 **팀 규칙을 자동으로 읽는다**

③ 팀마다 동일

실력 차이가 있어도 **구현 → 검증 → 커밋**의 같은 흐름으로 작업

규칙을 바꾸고 싶으면 Claude에게 **"CLAUDE.md의 ○○ 규칙을 수정해줘"**라고 말하면 끝.

추가 설치 없이 팀 작업 방식 고정 — 수업 시간은 기능 구현에 사용

CLAUDE.md 프로젝트 규칙 문서 · Claude에게 매번 줄 지시를 자동 적용 **git** version control · 코드 변경 이력을 저장·복원하는 도구

✧ SKILLS · 화면 품질 & 코드 다이어트

권고

프론트엔드 스킬 + ponytail

frontend-design ↗ 공식	**Anthropic 공식 디자인 스킬** — 템플릿 같지 않은 고유한 UI를 설계
taste-skill ↗	**"AI 티 나는" 밋밋한 디자인 방지** 스킬
getdesign.md ↗	실제 서비스 **300+의 디자인 명세** — 골라서 프로젝트 폴더에
21st.dev ↗	**UI·애니메이션 컴포넌트 갤러리** — 링크 붙여 "이 느낌으로"
ponytail ↗	**과잉 코드 방지** "게으른 시니어" 플러그인 — 불필요한 코드 증가를 막는다

전부 선택 사항 — 수업 중 설치하지 않는다. 시간이 남는 팀만 디자인 담당이 참고.

설치 자체도 "OO 설치해줘"로 가능 — 전체 커맨드는 발표자 메모(N)·공유 자료에

SECURITY · 보안 — 키 유출을 막는 한 장

비밀값은 .env, 제외 설정은 .gitignore



⚠ **하드코딩**하면 비밀값이 .env를 안 거치고 코드에 그대로 → **GitHub에 키 노출** → 봇이 몇 분 안에 도용

PROMPT · 배포 전 · GitHub 업로드 전 한 번씩

"점검해줘 — **API 키·비밀값이 코드에 노출된 곳** 있으면 전부 .env로 옮기고, **.gitignore에 .env가 등록**돼 있는지 확인해줘. 데모 데이터에 실제 개인정보가 있으면 가짜로 바꿔줘."

공개 저장소의 키 유출은 가장 흔한 보안 사고 — 이 한 장이 막는다

.env environment variables · 비밀값(키·비밀번호)을 코드와 분리해 담는 파일 **.gitignore** git ignore · 저장소에 올리지 않을 파일 목록(.env 등 보호)

team@hackathon:~$



Ⅳ · BUILD & SHIP

이기는 순서로 만들고, PDF와 MP4로 제출한다

PDF Portable Document Format · 인쇄·공유용 문서 형식

BUILD · 24시간 해커톤 타임라인

권고

핵심기능 3가지가 먼저 — 5단계

1	H0–1	**세팅 · 1시간 안에**	기획서 전달 → 개발 환경 확인 → 기능 ① 작업 시작
2	H1–6	**기능 ① 완성**	입력부터 결과까지 직접 눌러보고 완료 기준 통과
3	H6–11	**기능 ② 완성**	기능 하나에 집중 — 구현 → 검증 → 수정 → 커밋
4	H11–16	**기능 ③ 완성**	세 번째 기능도 같은 루프 — 구현 → 검증 → 수정 → 커밋
5	H16–24	**통합 + 제출 · 8시간**	기능 동결 → 전체 연결·검증 → IR 덱 PDF → 데모 영상 MP4

3기능 룰 **코어 1개 + 서포트 2개**를 하나씩 완성 — 경험 편차가 있어도 같은 완주 기준을 본다

마감 룰 **H16부터 새 기능 금지** — PDF·MP4는 제출 2시간 전에 완성하고 마지막 2시간은 재생·열기·업로드 확인

시장조사·기획·기능 선정은 완료 — 현장에서는 구현과 검증에 집중

PDF Portable Document Format · 인쇄·공유용 문서 형식

BUILD · 기획 완료 → 바로 구현

권고

준비된 3개를 하나씩 완성한다

기능마다 완료 기준 3문장

- ✓ **입력**: 사용자가 무엇을 누르고 입력하는가
- ✓ **결과**: 화면에 무엇이 보여야 성공인가
- ✓ **유지**: 새로고침·재실행 뒤에도 남아야 하는가

팀 공통 작업 원칙

- × 세 기능을 **한 번에 요청하지 않는다**
- × 기능 하나 끝날 때마다 **직접 눌러본다**
- × 완료되면 **커밋 + /clear**
- × 막히면 기능 추가 대신 **범위를 줄인다**
- × 잘하는 팀도 **네 번째 기능은 금지**

오늘은 다시 기획하지 않는다. 준비한 핵심기능 3가지를 **작동하는 상태로 완성**하는 것이 목표다.

BUILD · 작업 사이클

권고

해커톤 내내 이 루프만 반복한다



원칙: 한 번에 기능 **하나** — 3개를 한 번에 시키면 **70%짜리 3개**, 하나씩 시키면 **100%짜리 3개**. 끝나면 **커밋 + /clear** → 다음 기능. 1회 30분-1시간.

↑ SHIP · VERCEL · 해커톤은 끝나도 URL은 남는다

git push하면 — 자동 재배포

영상 속 **진짜 URL = 증거** · 끝나도 살아있는 **링크 = 사업의 시작** · 노트북이 죽어도 **서비스는 산다**

"Vercel로 배포해줘" — 나머지는 Claude가 자동으로



GitHub 이메일 인증 → Vercel에서 **Continue with GitHub** → Add New / Project → 저장소 **Import** → Deploy

최종 확인: 핵심기능 3가지 검증 → 보안 점검 → main에 push → Vercel 배포 완료 → 휴대전화에서 URL 확인.

GitHub 계정 안내 · Vercel × GitHub 공식 가이드

URL Uniform Resource Locator · 웹 주소 **git** version control · 코드 변경 이력을 저장·복원하는 도구

SHOW · 3분 공식 & 촬영

권고

3분 공식 — MP4로 제출

0:00–0:30	0:30–2:10	2:10–2:40
문제	**데모 — 핵심기능 3개를 하나의 여정으로**	**비전 + 팀**
구체적인 한 사람의 불편	기능 나열 ✕ → 한 사용자의 스토리 ○ · 주소창에 진짜 URL	어디까지 갈 것인가

비율 룰 데모 ≥ 50% — **문제 설명으로 절반을 소모하는 팀**이 가장 흔한 패배 패턴

촬영 룰 도구는 전부 기본 내장 — Mac ⌘⇧5 + **iMovie** · Win **Win+G + Clipchamp** · 해피 패스만, 2테이크

대본도 말로 — "PLAN.md 기준 3분 영상 대본 써줘, 화면 동선과 내레이션 나란히" · 상세 체크는 부록·메모에

PLAN.md 계획 문서 · 만들 기능과 범위를 미리 정리해 두는 파일 **URL** Uniform Resource Locator · 웹 주소

WRAP-UP · 처음의 그 다섯 줄

다시 — 오늘의 핵심 5

- ✓ **말하면, 만들어진다** — 단, 확인은 내 눈으로
- ✓ **세팅은 복사 한 줄** — 이미 여러분 노트북에서 첫 페이지가 나왔다
- ✓ **CLAUDE.md 한 장** — 설치 없이 팀 작업 규칙 자동 적용
- ✓ **비밀값은 .env로** — .gitignore가 키 유출을 막는다
- ✓ **핵심기능 3가지가 먼저** — IR 덱은 PDF · 데모 영상은 MP4

다섯 줄 전부 — **이제 여러분 것**이다.

시작 전 체크리스트·프롬프트 모음은 부록에 — 이 자료 전체 공유

.env environment variables · 비밀값(키·비밀번호)을 코드와 분리해 담는 파일 **.gitignore** git ignore · 저장소에 올리지 않을 파일 목록(.env 등 보호)

team@hackathon:~$ ▮

코드는 AI가,
창업은 여러분이

질문 받겠습니다 — 여러분의 해커톤을 응원합니다 🚀

APPENDIX

여기부터는 자료용

발표는 끝났습니다 — 해커톤 중에 펼쳐 보세요

APPENDIX · 해커톤 시작 전, 팀 단위로

시작 전 체크리스트

- ✓ 메인 노트북 — **Max 계정 로그인 + 첫 페이지**까지 재현 (오늘 한 것)
- ✓ 드라이버·내비게이터·디자인·영상 — **역할 지정**
- ✓ 완료한 **시장조사·서비스 기획서·핵심기능 3가지** 파일 준비
- ✓ 메인 노트북 프로젝트에 **CLAUDE.md 작업 규칙** 저장
- ✓ **GitHub 가입 + 이메일 인증** — github.com/signup
- ✓ **Vercel을 GitHub로 가입 + 저장소 권한 승인** — vercel.com/signup
- ✓ 앱 사용자는 **Claude 앱 설치 + 테스트 폴더 열기** — claude.com/download
- ✓ 영상 담당 — **녹화 단축키 테스트 1회** (Mac ⌘⇧5 · Win Win+G)

GitHub · Vercel · Claude 앱 — 공식 시작 페이지

CLAUDE.md 프로젝트 규칙 문서 · Claude에게 매번 줄 지시를 자동 적용

APPENDIX · 시작 직후 15분

기획 완료 — 개발 명세로 변환

PROMPT · 해커톤 시작 직후, 빈칸만 채워 전송

① **자료 전달** — 첨부한 시장조사·서비스 기획서와 확정된 **핵심기능 3가지**를 읽어줘.

② **PLAN.md로 변환** — 기능을 바꾸거나 추가하지 말고 구현 순서대로 저장

화면 목록 · 기능별 입력/결과/유지 조건 · 직접 눌러볼 검증 절차

③ **CLAUDE.md도 생성** — 팀 규칙

항상 한국어로 답하기 · git 저장소 만들고 기능 하나 끝날 때마다 커밋 · 새 도구는 내게 물어보기

✓ **새 기획·새 기능 제안은 받지 않는다** — 준비된 범위를 구현 언어로만 변환

✓ 이후 모든 기능 요청은 **"PLAN.md의 기능 ①을 만들자"**로 시작

범위는 "뭘 만들까"가 아니라 "뭘 안 만들까"로 지켜진다

CLAUDE.md 프로젝트 규칙 문서 · Claude에게 매번 줄 지시를 자동 적용 **PLAN.md** 계획 문서 · 만들 기능과 범위를 미리 정리해 두는 파일

APPENDIX · 명세 공식

좋은 요청 = 맥락 + 목표 + 제약 + 완료기준

- **맥락** — 어느 화면·어떤 상태에서
  "메인 페이지의 요청 목록에서"
- **목표** — 무엇이 되어야 하는가
  입력과 기대 출력
- **제약** — 건드리지 말 것·지킬 것
  "다른 기능은 그대로 둬"
- **완료 기준** — 내가 눌러 확인할 문장
  검증 단계와 맞물리는 톱니

EXAMPLE

[맥락] 메인 페이지에 산책 요청 목록이 있어.
[목표] 상단에 "요청 등록" 폼 추가 — 날짜·시간대·메모 입력, 저장하면 목록 맨 위에 바로 표시.
[제약] localStorage 사용, 기존 목록 디자인 유지.
[완료기준] 등록 → 목록 반영 → **새로고침해도 유지**.

APPENDIX · 상황별 즉전력

바로 쓰는 프롬프트 팩

에러가 났다

"이 에러를 고쳐줘. **원인부터 설명**하고 고쳐:
[에러 전체 붙여넣기 — 요약 금지]"

화면이 못생겼다

"**[스크린샷 첨부]** 이 화면을 더 신뢰감 있게 — 색·간격·글자 크기만 다듬고 **기능은 건드리지 마**"

기능이 끝났다

"방금 기능을 **내가 직접 확인할 방법**을 단계별로 알려줘 — 순서대로 눌러볼게"

공통 원칙 — 에러는 **전체를**, 화면은 **스크린샷으로**, 확인은 **내 손으로**.

APPENDIX · 막혔을 때

권고

막힘 대응 — 위에서부터 순서대로

1 · 에러가 보인다	**전체를 복사·붙여넣기** + "원인부터 설명하고 고쳐줘"
2 · 말로 설명이 어렵다	**스크린샷 붙여넣기** — 이미지가 천 마디 말보다 낫다
3 · 대화 맥락이 흐트러진다	Esc → `/clear` → **더 작은 단위로** 다시 요청
4 · 점점 더 망가진다	**"마지막 잘 되던 상태로 되돌리고**, 다른 방법으로 접근해줘"
5 · 같은 곳에서 30분+	그 기능을 **우회(함정 치환)**하거나 **멘토 호출**

막힘은 실력 문제가 아니라 "이 기능이 이번 해커톤 크기가 아니다"라는 신호일 수 있다

APPENDIX · 반면교사

권고

팀이 실패하는 방식 5가지

- × **로그인·결제·관리자부터 만들기** — 점수 없는 곳에 반나절을 붓는다
- × **검증 안 된 새 기술 도입** — 검증은 행사 전에 — 해커톤에선 익숙하고 검증된 스택만
- × **새벽 3시의 "전체 구조 정리"** — 피곤할수록 대규모 구조 변경 금지
- × **API 키를 코드에 하드코딩한 채 공개** — 키 유출 사고 · .env와 .gitignore로
- × **발표 준비 0분** — 기능 100% × 전달 0% = 0점

이 다섯만 피하면 — **중간 이상은 보장**된다.

.env environment variables · 비밀값(키·비밀번호)을 코드와 분리해 담는 파일 **.gitignore** git ignore · 저장소에 올리지 않을 파일 목록(.env 등 보호)

APPENDIX · 팀 자리에 붙여둘 것

해커톤 생존 규칙 6

- **확인 없이 다음 기능 없다** — 만들었다 + 눌러봤다 = 한 세트
- **한 번에 하나** — 70%짜리 셋보다 100%짜리 하나씩
- **배포는 선택** — 핵심기능 3가지가 모두 동작한 뒤 진행
- **대본에 없으면 만들지 않는다** — 영상 대본이 곧 범위
- **키는 코드에 넣지 않는다** — .env + .gitignore
- **제출은 여유 30분** — 마감 30초 전 업로드는 반드시 실패한다

새벽 3시의 나를 지키는 여섯 줄

.env environment variables · 비밀값(키·비밀번호)을 코드와 분리해 담는 파일 **.gitignore** git ignore · 저장소에 올리지 않을 파일 목록(.env 등 보호)

APPENDIX · 팀 병렬 작업 제안

심화 · 선택

더 빠르게? — 워크트리로 병렬

여러 명이 동시에 개발하고 싶다면 — **같은 프로젝트를 폴더 여러 개로 펼쳐** 각자 다른 기능을 **동시에** (사람도, AI 🤖도 폴더 하나씩)




```
# 같은 저장소를 옆 폴더로 하나 더 — 기능 B 전용 작업 공간
$ git worktree add ../featureB -b feature-b
$ cd ../featureB && claude # 그 폴더에서 에이전트 하나 더
```

선택 관리 도구 **orca**·**cmux**(Mac) · 각 워크트리 안에선 여전히 "한 번에 하나".

처음이면 한 명이 한 대에서(가장 안전) → 익숙해지면 워크트리로 병렬 · 6시간 룰 통과 후 권장

git version control · 코드 변경 이력을 저장·복원하는 도구

멱등성 (Idempotency) 이란?

같은 요청이 여러 번 와도 결과는 한 번만 반영되는 성질!

네트워크 문제나 사용자의 실수로 동일한 요청이 여러 번 전송될 수 있어요.
멱등성이 있으면, 아무리 여러 번 요청해도 결과는 한 번만 처리됩니다.



예시: 결제 요청



사용자가 결제 버튼을 빠르게 **두 번** 눌렀어요!

→ 서버로 동일한 결제 요청이 두 번 전송됩니다.

요청 1 →
요청 2 →
서버

멱등성이 없는 경우 (문제 발생!)

요청 1 처리 → 결제 1건 생성
요청 2도 다시 처리 → 결제 1건 또 생성
→ **결제 중복!** 원치 않는 결과 발생

멱등성이 있는 경우 (안전!)

요청 1 처리 → 결제 1건 생성
요청 2 도착 → 이미 처리된 요청인지 확인
→ 중복 처리 X 결과는 한 번만!

왜 멱등성이 필요할까요?

네트워크 문제
요청이 끊겼다가 다시 전송될 수 있어요.

버튼을 여러 번 누를 수 있어요.

서버/클라이언트 재시도
응답을 못 받으면 다시 요청할 수 있어요.

멱등성은 중복 처리로 인한 문제를 예방하여 **데이터의 정확성과 안정성을 지켜줍니다.**

어떻게 멱등성을 구현할까요?

1. **고유한 키 발급**
요청마다 고유한 멱등성 키를 함께 전송
Idempotency-Key

2. **요청 처리 및 저장**
요청을 처리하고, 결과를 키와 함께 저장

3. **중복 요청 확인**
같은 키로 요청이 오면 저장된 결과를 그대로 반환

같은 키로 들어온 요청은 새로 처리하지 않고, 기존 결과를 응답합니다.

한 줄 요약 멱등성 = 같은 요청이 여러 번 와도 **결과는 한 번!** → **안전하고 정확한 서비스**를 만들기 위한 기본 원칙입니다.

ACID 트랜잭션이란?

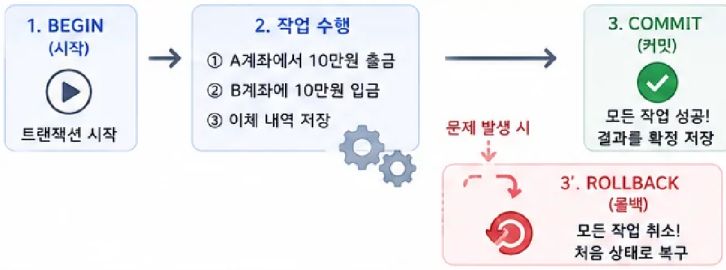
여러 작업을 하나의 단위로 묶어서,
모두 성공하거나 모두 실패하도록 보장하는 원칙!

예시: 계좌 이체 A계좌에서 B계좌로 10만원을 이체하는 과정





트랜잭션 동작 과정



한 줄 요약 ACID 트랜잭션은 "여러 작업을 하나의 단위로 안전하게 처리해서, 데이터의 정확성과 일관성을 보장하는 방법"이에요!

ACID의 4가지 원칙

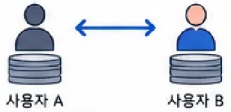
A Atomicity (원자성)
모든 작업이 성공하거나, 하나라도 실패하면 전체가 취소돼요.



C Consistency (일관성)
작업 전후에 데이터가 항상 올바른 상태를 유지해요.



I Isolation (독립성)
여러 작업이 동시에 실행돼도 서로 영향을 주지 않아요.



D Durability (지속성)
성공한 작업은 시스템이 꺼져도 결과가 영구히 저장돼요.



왜 ACID가 필요할까요?



데이터 손상 방지
중간에 오류가 나도 데이터가 깨지지 않아요.



동시 작업 안전
여러 사람이 동시에 사용해도 서로 영향을 주지 않아요.



신뢰할 수 있는 시스템
완료된 작업은 반드시 저장되고 사라지지 않아요.

신생활 비유
계좌 이체를 예로 들면,
출금, 입금, 기록 저장이 모두 성공해야 이체가 완료되고,
하나라도 실패하면 아무 일도 없던 것처럼 되돌려요!



YAGNI 원칙 (You Aren't Gonna Need It)



지금 필요하지 않은 기능은 미리 만들지 말자!

나중에 필요할지도 모르는 기능을 미리 만들면,
코드가 복잡해지고 유지보수도 어려워져요.

한 줄 요약

"지금 필요하지 않은 기능은
만들지 말고,
필요해졌을 때 만들자!"

YAGNI를 지키지 않은 경우 (미리 만들기)

나중에 쓸 수도 있을 것 같아서, 지금 필요하지 않은 기능까지 미리 만들어요.



- 회원 등급
- 포인트 시스템
- 쿠폰 기능
- 해외 결제
- 실시간 채팅
- 리뷰 기능
- 배송 예약
- 알림 기능
- ...

결과

- 코드가 복잡해져요
- 버그가 늘어나요
- 유지보수가 어려워요
- 개발 속도가 느려져요

YAGNI를 지킨 경우 (필요할 때 만들기)

지금 필요한 기능만 만들고, 필요해지면 그때 추가해요.



한재 필요한 기능만!

상품 주문
(핵심 기능)

필요해지면 그때!

- 해외 결제 (나중에)
- 포인트 시스템 (나중에)
- 리뷰 기능 (나중에)
- ...

결과

- 코드가 단순해요
- 버그가 줄어요
- 유지보수가 쉬워요
- 개발 속도가 빨라요

언제 적용할까?

명확한 요구사항이 있을 때
지금 필요한 기능이 분명할 때!

빠르게 결과를 내야 할 때
핵심 기능에 집중해서 빠르게 출시!

변화가 많은 프로젝트일 때
나중에 요구사항이 바뀔 수 있어요!

유지보수가 중요한 프로젝트일 때
단순한 구조가 유지보수를 쉽게 해요!

기억하기 쉬운 예시

	지금 할 일	나중에 할 일
쇼핑몰 만들기	✓ 상품 주문 ✓ 결제	• 해외 결제 • 포인트 적립 • 리뷰 시스템 ...
블로그 만들기	✓ 글 작성 ✓ 글 목록 보기	• 댓글 기능 • 좋아요 기능 • 실시간 알림 ...

핵심 포인트

- **지금 필요한 기능에만 집중하자!**
  핵심 기능을 빠르고 단순하게!
- **미래는 예측하기 어렵다!**
  지금 만들 기능이 정말 필요할지는 아무도 몰라요.
- **필요해졌을 때 만드는 것이 더 쉽다!**
  요구사항이 명확해진 후 만드는 것이 더 정확하고 효율적이에요.

YAGNI = 단순하게 시작하고, 필요할 때 발전시키자!

APPENDIX · 실제 터미널 예시

참고

실제 화면 — 터미널에서 이렇게 대화

입력창에 **말**을 적으면 → AI가 **계획하고 파일을 직접 고친다** → 확인 방법까지.

team-cafe — claude

```
max@hackathon ~/team-cafe (main)

> 메인 페이지에 "문의하기" 버튼 추가해줘.
  누르면 폼이 열리고, 제출하면 아래 목록에 뜨게. 기존 디자인 유지.

● 좋아요. 먼저 페이지 구조를 볼게요.
  ▸ Read  app/page.tsx  ⎿ 120줄

● 버튼 + 폼(모달) + 목록을 추가하겠습니다.
  ▸ Edit  app/page.tsx  ⎿ +38 −2  → 적용할까요? ❯ Yes

● 완료! 버튼을 누르면 폼이 열리고, 제출하면 목록에 바로 나타나요.
  ▸ 확인: 버튼 클릭 → 내용 입력 → 제출 → 목록에 뜨는지 눌러보기

> ▌
```

① 말한다 — 구체적으로
맥락 + 목표 + 제약 + 완료기준

② AI가 계획·수정 — ▸ 는 도구 사용
파일 읽기(Read)·고치기(Edit)

③ 위험한 변경은 물어봄
읽고 ❯ Yes(Enter)로 허락

④ 확인 방법까지 알려줌
직접 눌러보고 → 다음 요청

실제 Claude Code 화면 재현(목업) · 기호: ● AI 말 · ▸ 도구 사용 · ⎿ 결과 · ❯ 선택(Enter) · > 내 입력 — 실제 스크린샷 있으면 이 자리에 넣기

APPENDIX · 무슨 스택으로 시작?

권고

스택 뭐 쓰지? — AI가 제일 잘 아는 걸로

고민하지 말고 **인기 스택**. 학습데이터가 많을수록 **AI 정답률↑ · 디버깅↓ · 빠름** · 잘 모르는 스택은 AI가 부정확한 코드를 내어 직접 수정할 일이 늘어남

★ 기본 경로 — 웹 (거의 항상 이거)

Next.js
화면+서버

프론트+API **한 몸** · TS 한 언어 · **AI가 제일 잘 씀**

Supabase
DB·로그인

DB·로그인·저장소 **즉시** · 단순 CRUD면 백엔드 **0줄**

Vercel
배포

git push = 자동 배포 · 링크 하나로 폰에서 바로 연다

고민되면 이 3질문만

❓ 앱이야 웹이야?	→ **웹** — 데모 = URL 하나, 속도 압승
❓ 파이썬 꼭 필요? (ML·데이터·크롤링)	→ 그때만 **+ FastAPI** 백엔드로 · 아니면 그대로
❓ 기기 기능 꼭? (카메라·푸시·오프라인)	→ 그때만 **네이티브 앱** (React Native/Flutter) · 아니면 웹

한 줄: 고민 말고 `Next.js + Supabase + Vercel` · 파이썬 꼭이면 그때만 `FastAPI` · 기기기능 꼭이면 그때만 **앱** · 원칙은 "AI가 잘 아는 인기 스택" · 2026-07 기준

URL Uniform Resource Locator · 웹 주소 **API** Application Programming Interface · 프로그램끼리 데이터를 주고받는 통로

APPENDIX · DNS → VERCEL → SERVERLESS → DATABASE

나중에 이렇게

진짜 서비스는 이 아키텍처로



DNS · 도메인

루트 도메인은 **A 레코드**, 서브도메인은 **CNAME**으로 Vercel 연결 · 검증 후 SSL 자동 발급

Vercel · 서버리스

정적 화면은 CDN, 서버 코드는 요청마다 함수로 실행 · **함수와 DB 리전을 가깝게**

Supabase · DB

서버리스 연결은 **transaction pooler** · 모든 공개 테이블은 **RLS**

비밀값 경계: `service_role`·DB 비밀번호는 **Vercel 환경변수에만** — 브라우저 코드·GitHub·`NEXT_PUBLIC_*`에 넣지 않는다.

Vercel DNS · Vercel Functions · Supabase DB 연결 · Supabase 운영 체크리스트

APPENDIX · PROTOTYPE → PRODUCTION

나중에 이 순서로 실제 배포

01 · DB
Supabase 생성
함수와 가까운 리전 · Auth·Storage 설정

→

02 · SCHEMA
Migration 배포
db reset으로 검증 후 db push

→

03 · SECURITY
RLS 설정
테이블별 읽기·쓰기 정책과 권한 테스트

→

04 · APP
Vercel 배포
환경변수 등록 → main을 Production으로

```
# DB 스키마도 코드처럼 버전 관리
$ supabase migration new create_core_tables
$ supabase db reset # 로컬 재현·검증
$ git add supabase/migrations && git commit
$ supabase link
$ supabase db push # 운영 DB 반영
```

- ✓ **Production 환경변수** 등록 후 재배포
- ✓ **main**만 운영 도메인에 연결
- ✓ 도메인 추가 → **A/CNAME** 설정
- ✓ **SSL·로그·권한·CRUD** 최종 확인

운영 원칙: 코드 배포와 DB 배포는 별개다. **DB migration 먼저 → 앱 배포 다음**. 운영 DB를 대시보드에서 즉흥 수정하지 않는다.

DB migration · Vercel 환경변수 · Local·Preview·Production · 커스텀 도메인

git version control · 코드 변경 이력을 저장·복원하는 도구

APPENDIX · CLAUDE 모델 고르기

권고

어떤 모델을 언제? — 그리고 바꾸는 법

Haiku 4.5

가장 **빠르고 저렴** · 간단·반복(요약·분류·자잘한 수정)

$1 / $5 · 200K

Sonnet 5 ★ 행사 추천

속도·코딩·사용량 균형 · 대부분의 작업은 여기서 시작

$2 / $10* · 1M

Opus 5

더 깊은 **추론** · Sonnet으로 풀리지 않는 복잡한 문제에 선택

$5 / $25 · 1M

Fable 5

최상위 **추론** · 대부분 과함 — **정말 안 풀릴 때만**

$10 / $50 · 1M

🐇 빠르고 저렴 ——— 똑똑하고 한도 많이 씀 🧠

```
# Claude 입력창에서 언제든 — 모델 바꾸기
/model # 목록에서 Sonnet 5 선택
/usage # 팀 공유 한도 확인
```

행사 추천은 Sonnet 5 · Anthropic 공식 안내 · *API 도입 가격은 2026-08-31까지

API Application Programming Interface · 프로그램끼리 데이터를 주고받는 통로

PROFILE · 오늘의 발표자



서외구 Max

Full-Stack Developer

seogolf665@gmail.com · 010-7320-7733

질문은 해커톤 동안 언제든